

Lab 6

In this lab, you will develop a program to analyze something like the “Facebook graph”, which stores information about people and the friendships that connect them.

Friends

Facebook allows you to invite other users to be your “friends”. If the other user accepts your invitation, then the system records that the two of you are friends, at which point you are given greater access to each other’s online personas. One way Facebook could represent these relationships is by listing each pair of friends. For example, consider the following input:

```
Alice Bob
Carol Alice
Bob Carol
Bob Doug
```

In this network, Alice, Bob, and Carol are mutual friends. Bob and Doug are also friends. Note that friendship is a symmetric relationship: Alice is Bob’s friend so Bob is also Alice’s friend even though only one direction of the relationship is explicitly mentioned.

Unzip `lab6given.zip`, available from the course webpage, and open this project by clicking on its file `lab6-friends.code-workspace`. It is a code fragment that opens a file (named “smallNetwork”) so that it can read using the `Scanner` called `input`. We’ll be expanding this code in steps so that it can read input formatted as above from this `Scanner` and create a data structure representing these relationships.

As a first step, add a `Set<String>` to the `Friends` class and read in the input. For each name you read, add it to the set. (Recall that `Scanner` has a `next` method that returns the next word of input.) Then add a method `summarize` that just prints the number of people seen (i.e. the number of strings in the set; use the `size` method of `Set`).

Once that is working, we want to store additional information. Create a `Person` class with a `String` attribute to store their name and a `List<String>` to store a list of their friends. For simplicity, make both of these attributes `public` rather than `private` like we’d normally make them. Give it a constructor that takes the name, setting that attribute and also giving the object an empty list of friends. (You can use either an `ArrayList` or a `LinkedList` for the list.) You also want to write a method `print` for `Person` that will print the information about that person in the following format:

```
Alice: Bob Carol
```

This the output for a `Person` whose name is `Alice` and whose friends are `Bob` and `Carol`. You will need a loop to print all the friends; remember how we used `foreach` loops or iterators to visit all the data items in a list.

Test the `Person` class by creating one in `main`, adding a couple of friends and then calling the `print` method.

Once the `Person` class works, change the set in `Friends` to a `Map<String, Person>`, which we’ll use to look up the `Person` with a given name. (In the `Friends` constructor, you’ll need to create either a `TreeMap` or a `HashMap`; it doesn’t matter which you use.) Then change the `Friends` constructor to read a pair of names at a time (2 separate calls to `next`). The constructor should make sure that each `Person` whose name you read exists and then put each name into the other name’s `Person` object. This makes the two people friends. The methods of `Map` that you’ll need for these operations are `containsKey` (to check if a `Person` already exists), `get`, and `put`.

Once you think you’ve read in the input and created the data structure, you need to modify the `summarize` method to list all the friends of each individual. To iterate through the keys of a map, you’ll want something like the following:

```
Iterator<String> key_it = map.keySet().iterator();
while(key_it.hasNext()) {
    String key = key_it.next();
    Person p = map.get(key);
    ...
}
```

Using this example, adapt `summarize` to give output formatted as follows (the order doesn’t matter):

```
Alice: Bob Carol
Bob: Alice Carol Doug
Carol: Alice Bob
Doug: Bob
```

Use the `print` method of `Person` to print each line.

The input file `smallNetwork` is included in the project. A larger example is available in `largerNetwork`.

Once you are confident that your program works, see if you can extend it to identify all users connected to Alice by chains of friendships. For example, in `largerNetwork` Jose should appear in the list because Alice is friends with Carol, who is friends with Gwen, who is friends with Jose. You can do this using depth-first search like we did last week, but I suggest the queue-based version of this instead. It’s called breadth-first search (BFS). To use it, put the Alice object into a queue and then repeatedly remove an element from the queue and add its friends to the queue. (The easiest way to do this is for the list of friends in `Person` to be public. We’d normally try to avoid this, but go with this here in the name of expediency.) Two notes about this:

1. You’ll want to add a `boolean` to each `Person` that indicates whether that object has been added to the queue already. The variable should start as false before being changed to true when the object is enqueued. Then, when you go through the list of friends, only enqueue the ones that haven’t already been put in the list.
2. To get a queue in Java, create a `LinkedList` and use the methods `offer` (“enqueue” in our terminology) and `poll` (“dequeue”).

If you have additional time, try to expand your analysis of the relationships by counting the number of components¹ or identifying the largest one.

¹A *component* is all the people connected together by a chain of friendships.